



Oracle Database Administration

I

Chapter 9: Schema Objects: Indexes, Views, Synonyms, and Sequences

محتويات الفصل:

Objectives	2
1. Deciding When to Create an Index?	3
2. Types of Indexes	5
2.1. B-tree index	5
2.2. Bitmap index	6
3. Creating Separate Tablespaces for Indexes	8
4. Creating Indexes	9
5. Maintaining Indexes	13
6. Implementing Views	15
7. Managing Synonyms	21
8. Managing Sequences	23
9. Quiz	27
References	33

Objectives:

1. Create indexes
2. Create views
3. Create Synonyms
4. Create sequences

1. Deciding When to Create an Index?

What is index

An index is an optionally created database object used primarily to increase query performance. Indexes can also limit the amount of data that is returned in the results without having to retrieve all of data columns of a table. This can help keep results in memory and bring back results faster.

Without indexes, the entire table is scanned (full table scan) with each DML operation on the table. Indexes are created automatically when a primary or unique keys are defined. Indexes allow for excellent scalability, even with very large data sets.

When to create indexes

If indexes are so important to database performance, why not place them on all tables and column combinations? The answer is short: indexes are not free. They consume disk space and system resources. As column values are modified, any corresponding indexes must also be updated. In this way, indexes use storage, I/O, CPU, and memory resources. A poor choice of indexes leads to wasted disk usage and excessive consumption of system resources. This results in a decrease in database performance and greater cost for DML statements.

There are usually two different situations in which DBAs and developers decide to create indexes:

- Proactively, when first deploying an application; the DBAs/developers make an educated guess as to which tables and columns to index.
- Reactively, when application performance bogs down, and users complain of poor performance

Indexes in oracle

Oracle allows you to create an index that contains more than one column. Multicolumn indexes are known as concatenated indexes (also called composite indexes). These indexes are especially effective when you often use multiple columns in the `WHERE` clause when accessing a table.

Columns included in the `SELECT` and `WHERE` clauses are also potential candidates for indexes. Sometimes, a covering index in a `SELECT` clause results in Oracle using the index structure itself (and not the table) to satisfy the results of the query. Also consider creating indexes on columns used in the `ORDER BY`, `GROUP BY`, `UNION`, and `DISTINCT` clauses. This may result in greater efficiency for queries that frequently use these SQL constructs.

2. Types of Indexes:

2.1. B–tree index:

Definition of B–tree index

Default index; good for columns with high cardinality (i.e., high degree of distinct values). Use a normal B–tree index unless you have a concrete reason to use a different index type or feature.

Structure of B–tree index:

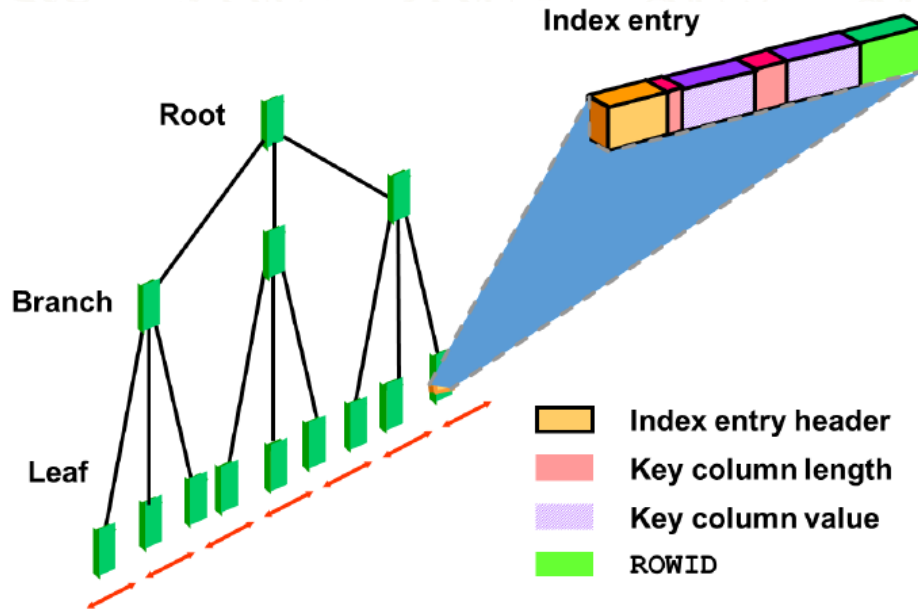
At the top of the index is the root, which contains entries that point to the next level in the index.

- At the next level are branch blocks, which in turn point to blocks at the next level in the index.
- At the lowest level are the leaf nodes, which contain the index entries that point to rows in the table.
- The leaf blocks are doubly linked to facilitate the scanning of the index in an ascending as well as descending order of key values.

Format of Index Leaf Entries

An index entry has the following components:

- Entry header: Stores the number of columns and locking information
- Key column length–value pairs: Define the size of a column in the key followed by the value for the column (The number of such pairs is the maximum of the number of columns in the index.)
- ROWID: Row ID of a row that contains the key values



When to use B-Tree indexes

The cardinality (the number of distinct values) in the column is high

- The number of rows in the table is high, and
- The column is used in `WHERE` clauses or `JOIN` conditions

2.2. Bitmap index:

Definition of Bitmap index

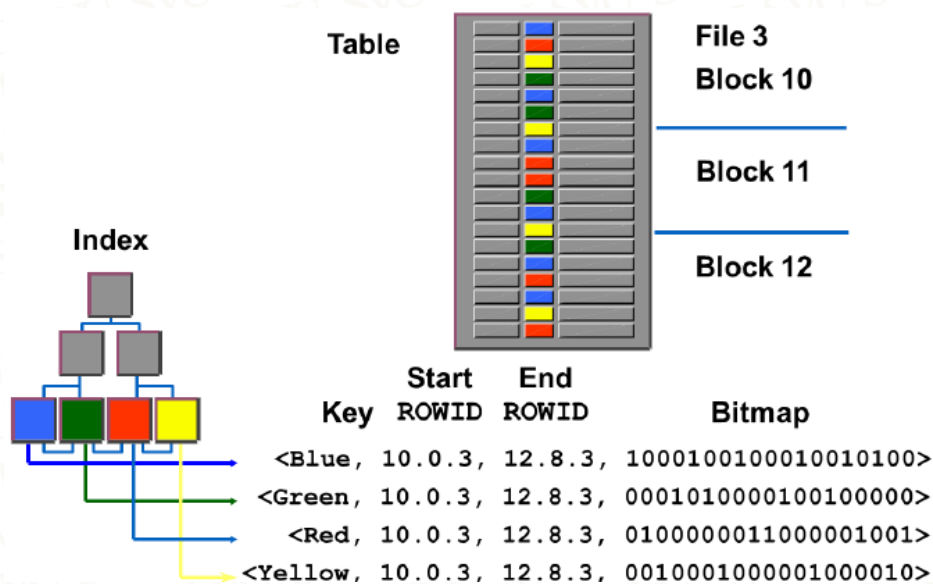
Excellent in data warehouse environments with low cardinality (i.e., low degree of distinct values) columns and SQL statements using many `AND` or `OR` operators in the `WHERE` clause. Bitmap indexes are not appropriate for OLTP databases in which rows are frequently updated. You cannot create a unique bitmap index.

Structure of a Bitmap Index

A bitmap index is also organized as a B-tree, but the leaf node stores a bitmap for each key value instead of a list of row IDs. Each bit in the bitmap corresponds to a possible row ID, and if the bit is set, this means that the row with the corresponding row ID contains the key value.

As shown in the diagram, the leaf node of a bitmap index contains the following:

- An entry header that contains the number of columns and lock information.
- Key values consisting of length and value pairs for each key column (In the slide example, the key consists of only one column; the first entry has a key value of Blue).
- Start ROWID, which in the example specifies block number 10, row number 0, and file number 3
- End ROWID, which in the example specifies block number 12, row number 8, and file number 3
- A bitmap segment consisting of a string of bits (The bit is set when the corresponding row contains the key value and is unset when the row does not contain the key value. The Oracle server uses a patented compression technique to store bitmap segments.)



When to use Bitmap indexes

- The cardinality (the number of distinct values) in the column is low
- The number of rows in the table is high
- The column is used in Boolean algebra operations

3. Creating Separate Tablespaces for Indexes:

Space consumption and object growth have a direct impact on database availability. If you run out of space, your database will become unavailable. The best way to manage space in the database is by creating tablespaces tailored to space requirements and then creating objects in specified tablespaces that you have designed for those objects. It is recommended that you separate tables and indexes into different tablespaces. Consider the following reasons:

- Doing so allows for differing backup and recovery requirements
- Tables and indexes often have different storage requirements (such as extent size and logging)
- When running maintenance reports, it is sometimes easier to manage tables and indexes when the reports have sections separated by tablespace
- Doing so increase the performance of actions especially, when the different tablespaces are on separate controllers

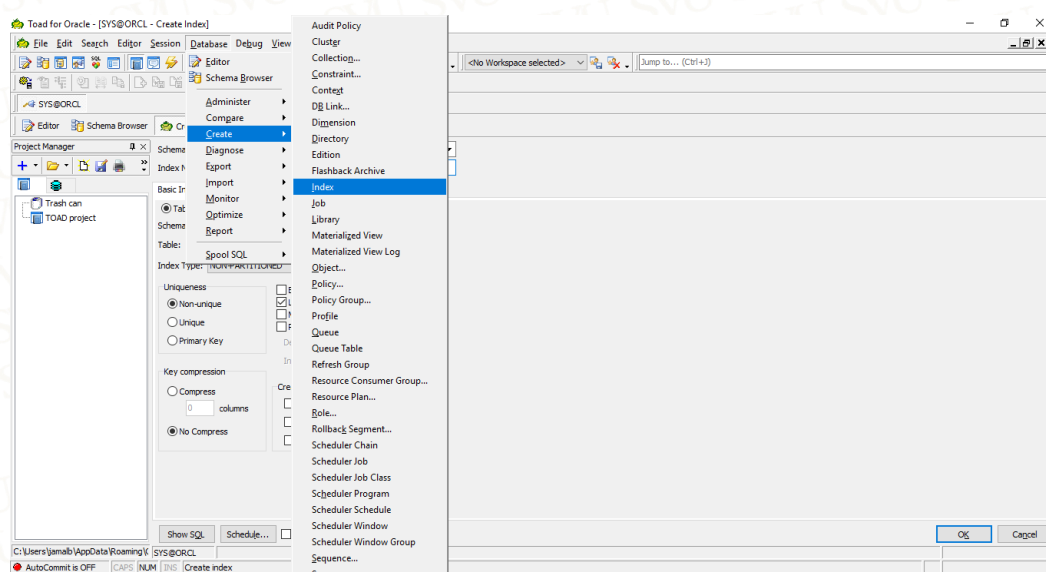
4. Creating Indexes:

Creating B-tree Indexes

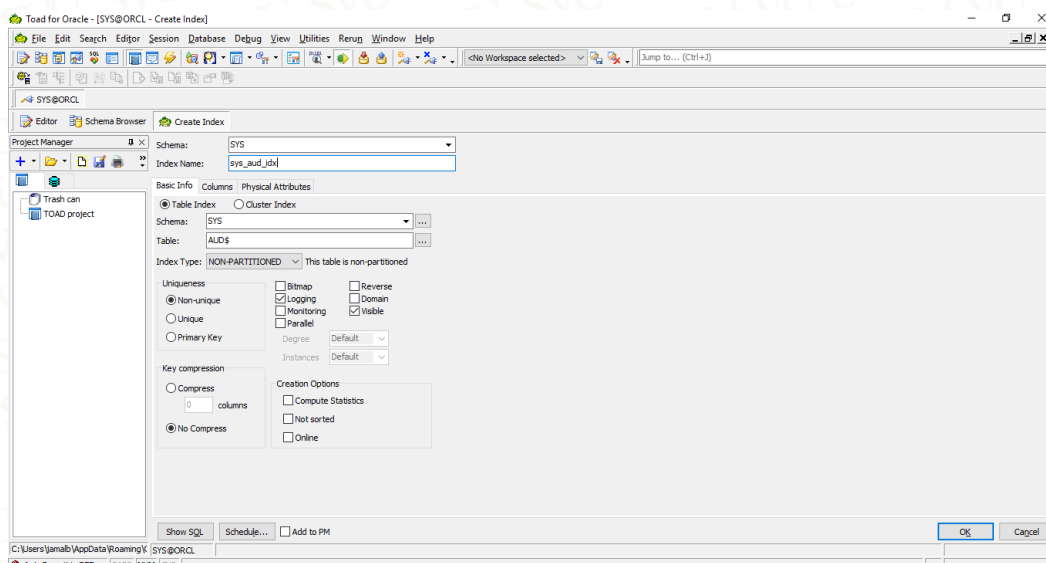
Creating indexes on the wrong columns or using features in the wrong situations can cause dramatic performance degradation.

To create index in Toad of Oracle please follow steps in this video

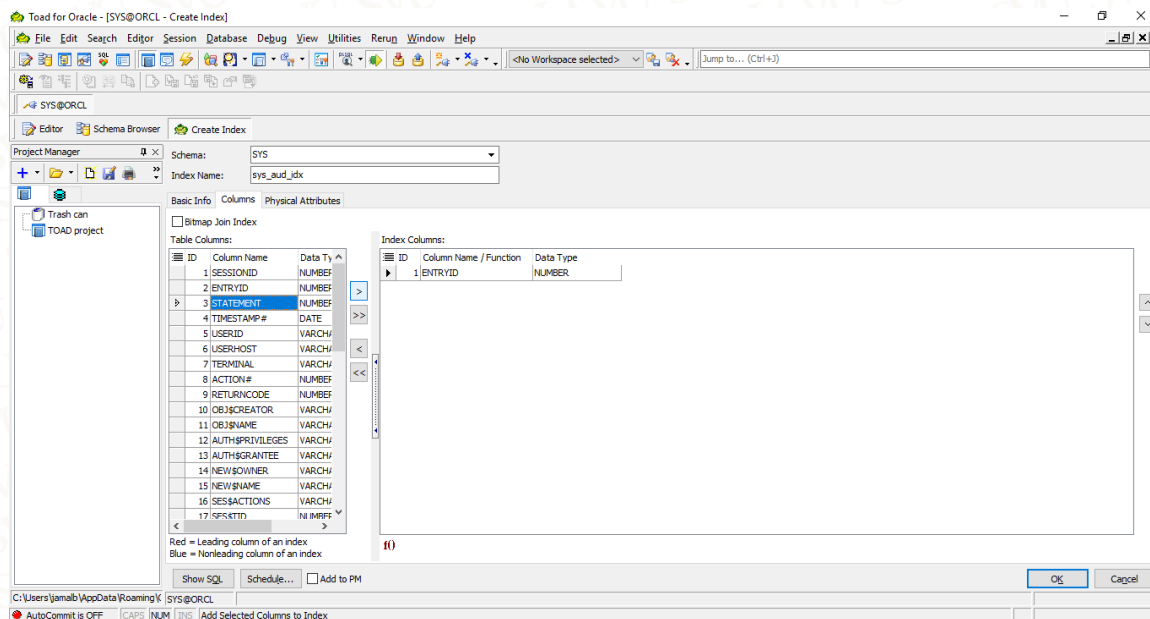
Choose create index from menu as the following:



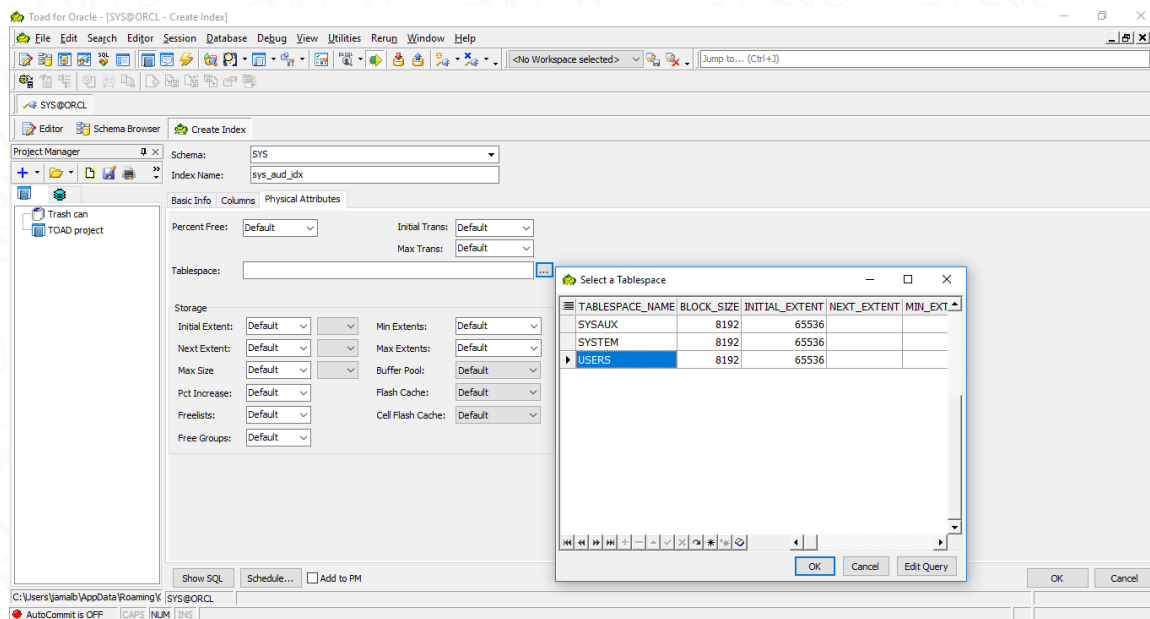
Then you enter the name of the index and its schema beside the basic info including the tables name and type of index:



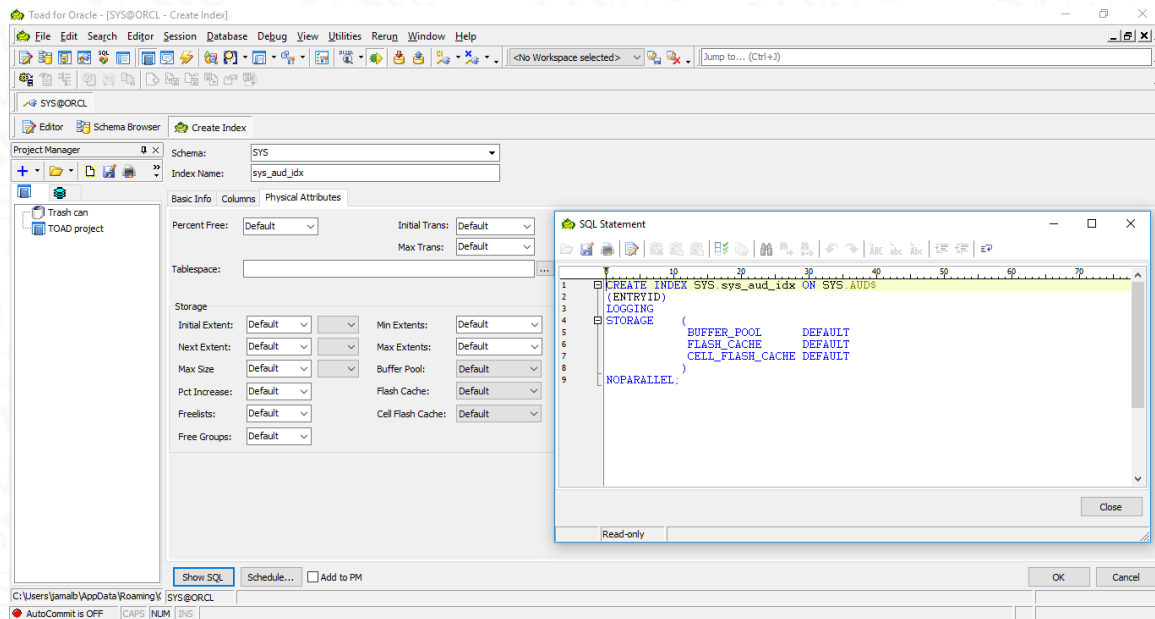
Then you enter the columns of table that related to the new index:



Then you enter the Physical Attributes that related to the new index:



Finally, you can see the SQL statement that related to create the new index:



To create a B-tree index on an existing table, use the `CREATE INDEX` statement:

```
SQL> CREATE INDEX cust_idx1 ON cust(last_name);
```

Oracle will create a B-tree index as default index type in your default permanent tablespace. For manageability reasons, you want to create the index in a specific tablespace:

```
SQL> CREATE INDEX cust_idx1 ON cust(last_name) TABLESPACE reporting_index;
```

Oracle provides two types of views containing details about the structure of B-tree indexes:

- `INDEX_STATS`
- `DBA/ALL/USER_INDEXES`

The `INDEX_STATS` view contains information regarding the `HEIGHT` (number of blocks from root to leaf blocks), `LF_ROWS` (number of index entries), and so on. The `INDEX_STATS` view is only populated after you analyze the structure of the index; for example,

```
SQL> analyze index cust_idx1 validate structure;
```

The `DBA/ALL/USER_INDEXES` views contain statistics, such as `BLEVEL` (number of blocks from root to branch blocks; this equals `HEIGHT-1`); `LEAF_BLOCKS` (number of leaf blocks); and so on. The `DBA/ALL/USER_INDEXES` views are populated automatically when the index is created and refreshed via the `DBMS_STATS` package.

Creating Concatenated Indexes

Oracle allows you to create an index that contains more than one column, Multicolumn indexes are known as concatenated indexes. These indexes are especially effective when you often use multiple columns in the `WHERE` clause when accessing a table.

Also, there can be only one visible index for the same combination of columns. Any other indexes created on that same set of columns must be declared invisible; for example:

```
SQL> create index cust_idx2 on cust(first_name, last_name);
```

```
SQL> create bitmap index cust_bmx1 on cust(first_name,  
last_name) invisible;
```

Creating Unique Indexes

Use the `UNIQUE` clause to create a unique index:

```
SQL> create unique index cust_uk1 on cust(first_name,  
last_name);
```

When you create a primary key constraint or a unique key constraint, Oracle automatically creates a unique index and a corresponding constraint that is visible in `DBA/ALL/USER_CONSTRAINTS`.

You have to look at `DBA_INDEXES` and `DBA_IND_COLUMNS` to view the details of the unique index that has been created:

```
SQL> select a.owner, a.index_name, a.uniqueness, b.column_name  
from dba_indexes a, dba_ind_columns b  
where a.index_name='CUST_UK1'  
and a.table_owner = b.table_owner  
and a.index_name = b.index_name;
```

5. Maintaining Indexes:

Renaming an Index

Use the ALTER INDEX ... RENAME TO statement to rename an index:

```
SQL> alter index cust_idx1 rename to cust_index1;
```

Displaying the DDL for an index

You can use the DBMS_METADATA package to display the DDL required to re-create an Index:

```
SQL> set long 10000
SQL> select dbms_metadata.get_ddl('INDEX','CUST_IDX1') from
dual;
```

--Here is a partial listing of the output:

```
SQL> CREATE INDEX "MV_MAINT"."CUST_IDX1" ON "MV_MAINT"."CUST"
("CUST_ID")
PCTFREE 10 INITRANS 2 MAXTRANS 255 INVISIBLE COMPUTE STATISTICS
```

To show all index DDL for a user, run this query:

```
SQL> select dbms_metadata.get_ddl('INDEX',index_name) from
user_indexes;
```

Rebuilding an Index

Use the REBUILD clause to rebuild an index:

```
SQL> alter index cust_idx1 rebuild;
```

Oracle attempts to acquire a lock on the table and rebuild the index online. If there are any active transactions that haven't committed, Oracle will not be able to obtain a lock, and the following error will be thrown:

```
ORA-00054: resource busy and acquire with NOWAIT specified or
timeout expired
```

You can either wait until there is little activity in the database or try setting the DDL_LOCK_TIMEOUT parameter:

```
SQL> alter session set ddl_lock_timeout=15;
```

Making Indexes Unusable

If you have identified an index that is no longer being used, you can mark it `UNUSABLE`. The advantage of marking the index `UNUSABLE` (rather than dropping it) is that if you later determine that the index is being used, you can alter it to a `USABLE` state and rebuild it without needing the DDL on hand

Here is an example of marking an index `UNUSABLE`:

```
SQL> alter index cust_idx1 unusable;
```

You can verify that it is unusable via this query:

```
SQL> select index_name, status from user_indexes;
```

If you determine that the index is needed (before you drop it), then it must be rebuilt to become usable again:

```
SQL> alter index cust_idx1 rebuild;
```

Monitoring Index Usage

Use the `ALTER INDEX...MONITORING USAGE` statement to enable basic index monitoring:

```
SQL> alter index cust_idx1 monitoring usage;
```

To report which indexes are being monitored and have been used, run this query:

```
SQL> select * from dba_index_usage;
```

Dropping an Index

Use the `DROP INDEX` statement to drop an index:

```
SQL> drop index cust_idx1;
```

6. Implementing Views:

Views are representations of queries of data from one or more tables or other views. Views are stored queries because they can hide very complex conditions and joins as well as other complex expressions and SQL constructs. Views do not actually contain data; instead, they derive their data from the tables on which they are based. These tables are referred to as the base tables of the view.

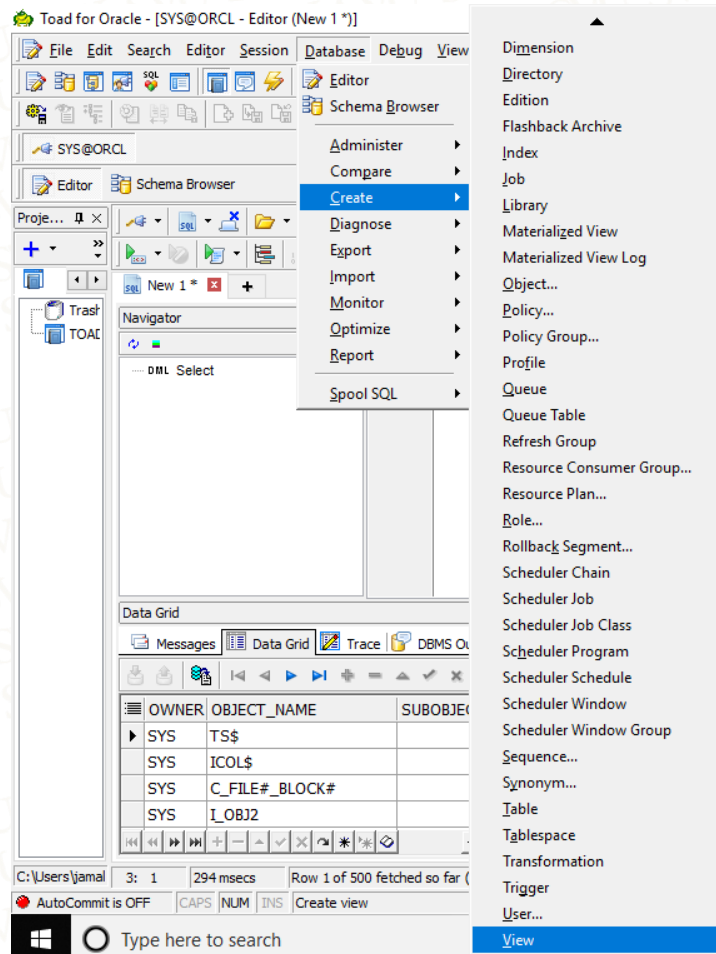
Like tables, views can be queried, updated, inserted into, and deleted from—with some restrictions. All operations performed on a view actually affect the base tables of the view. Views provide an additional level of security by restricting access to a predetermined set of rows and columns of a table. They also hide data complexity and store complex queries. The common uses for views:

- Create an efficient method of storing an SQL query for reuse
- Provide an interface layer between an application and physical tables
- Hide the complexity of an SQL query from an application
- Report to a user only a subset of columns or rows, or both

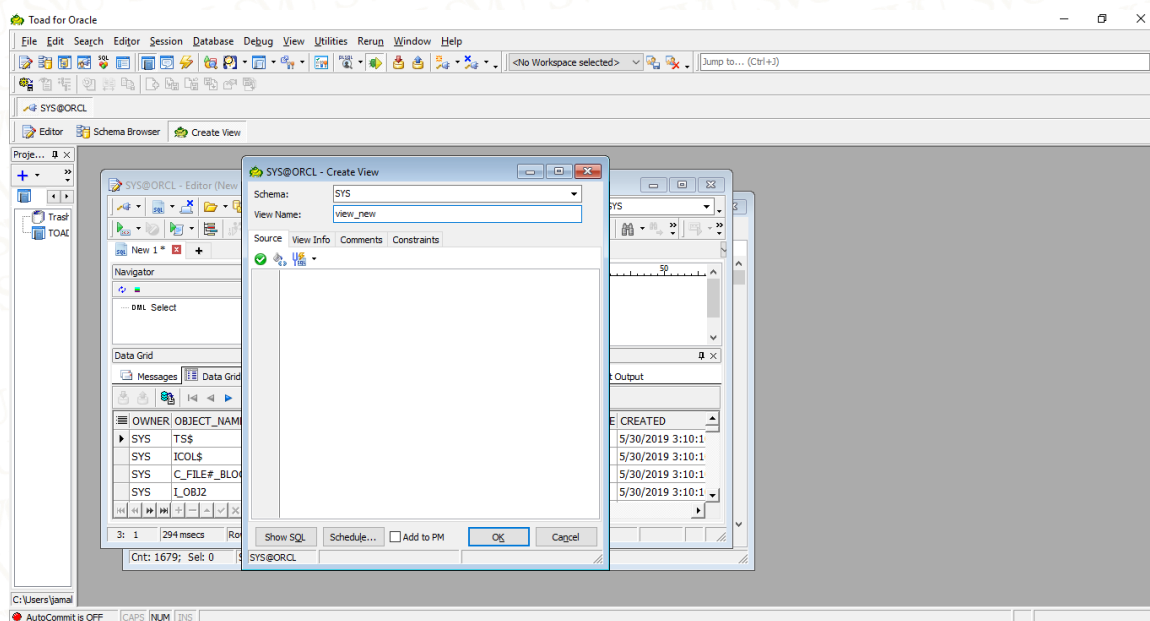
Creating a View

To Create view in Toad of Oracle please follow the steps shown in the [video](#)

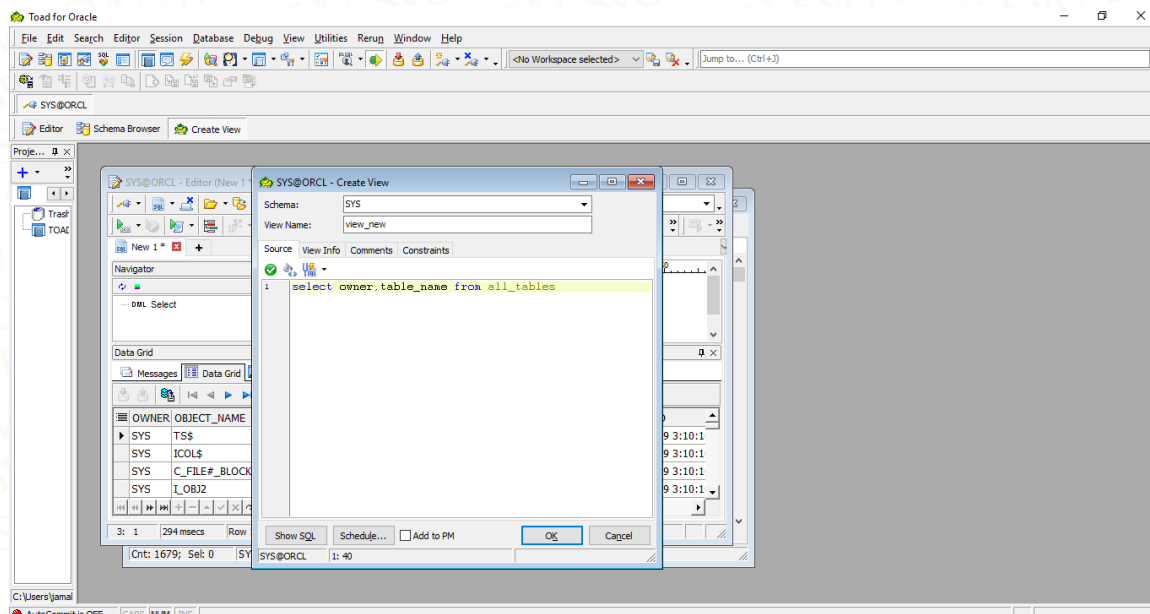
Choose create view from menu as the following:



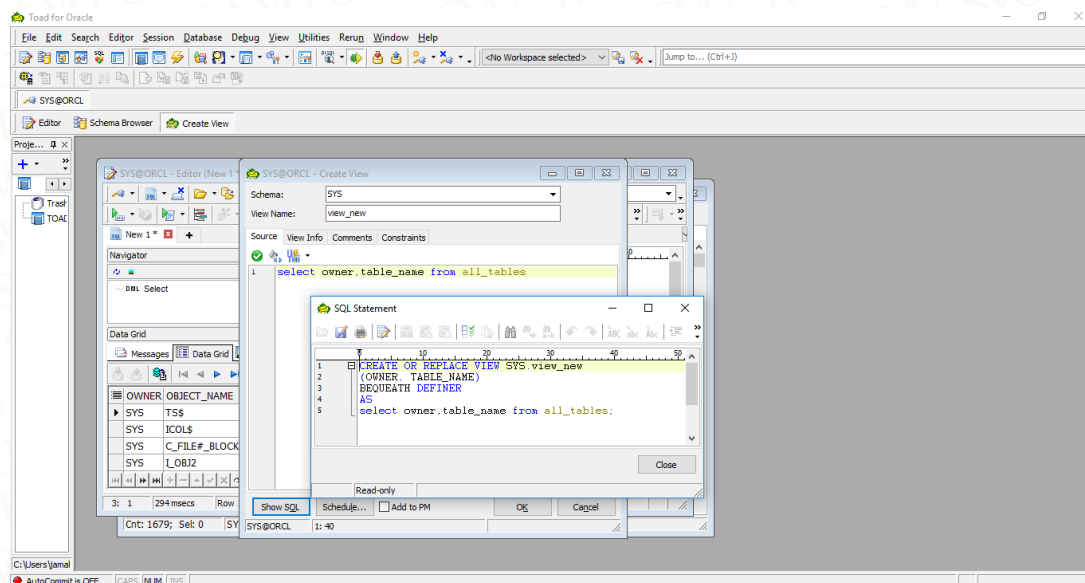
Then you can enter the schema name and the view name and the select statement – step1:



Then you can enter the schema name and the view name and the select statement –
step2:



Finally, you can see the statement of creating the new view the:



Or, you can create views directly as follows:

Assume that we have the following base table:

```
SQL> create table sales(sales_id number primary key , amnt
number, state varchar2(2), sales_person_id number);
```

Also assume that the table has the following data initially inserted into it:

```
SQL> insert into sales values(1, 222, 'CO', 8773);
SQL> insert into sales values(20, 827, 'FL', 9222);
```

Then, the `CREATE VIEW` statement is used to create a view. The following code creates a view (or replaces it if the view already exists) that selects a subset of columns and rows from the `SALES` table:

```
SQL> create or replace view sales_rockies as
select sales_id, amnt, state
from sales
where state in ('CO','UT','WY','ID','AZ');
```

When you select from `SALES_ROCKIES`, it executes the view query and returns data from the `SALES` table as appropriate:

```
SQL> select * from sales_rockies;
```

Given the view query, it is intuitive that the output shows only the following columns and one row:

SALES_ID	AMNT	ST
-----	-----	--
1	222	CO

The following insert statement against the view results in the insertion of a record in the `SALES` table:

```
SQL> insert into sales_rockies(sales_id, amnt, state)
values (2,100, 'CO');
```

You can grant DML privileges to other users on the view. For instance, you can grant `SELECT`, `INSERT`, `UPDATE`, and `DELETE` privileges on the view to another user, which will allow the user to select and modify data referencing the view:

```
SQL> insert into sales_rockies(
sales_id, amnt, state)
values (3,123, 'CA');
```

```
SQL> select * from sales_rockies;
SALES_ID AMNT ST
-----
1 222 CO
2 100 CO
```

the query on the underlying table shows that rows exist that are not returned by the view:

```
SQL> select * from sales;
SALES_ID      AMNT      ST      SALES_PERSON_ID
-----
1           222      CO           8773
20          827      FL           9222
2           100      CO
3           123      CA
```

Creating Read-Only Views

If you do not want a user to be able to perform INSERT, UPDATE, or DELETE operations on a view, then do not grant those object privileges on the view to that user. Furthermore, you should also create a view with the WITH READ ONLY clause for any views for which you do not want the underlying tables to be modified. The default behavior is that a view is updatable (assuming the object privileges exist).

This example creates a view with the WITH READ ONLY clause:

```
SQL> create or replace view sales_rockies as
select sales_id, amnt, state
from sales
where state in ('CO','UT','WY','ID','AZ')
with read only;
```

Displaying the SQL Used to Create a View

To display the text associated with a particular view for a user:

```
SQL> select view_name, text
from dba_views
where owner = upper('&owner')
and view_name like upper('&view_name');
```

You can also query `ALL_VIEWS/USER_VIEWS` for the text of any view you have access to:

```
SQL> select text
from all_views
where owner='MV_MAINT'
and view_name='SALES_ROCKIES';
```

Dropping a View

Before you drop a view, consider renaming it. If you are certain that a view is not being used anymore, then it makes sense to keep your schema as clean as possible and drop any unused objects. Use the `DROP VIEW` statement to drop a view:

```
SQL> drop view sales_rockies_old;
```

7. Managing Synonyms:

Synonyms provide a mechanism for creating an alternate name or alias for an object.

You can create synonyms for the following types of database objects:

- Tables
- Views, object views
- Other synonyms
- Remote objects via a database link
- PL/SQL packages, procedures, and functions
- Materialized views
- Sequences
- Java class schema object
- User-defined object types

Creating a Synonym

A user must be granted the `CREATE SYNONYM` system privilege before creating a synonym. Once that privilege is granted, use the `CREATE SYNONYM` command to create an alias for another database object.

For example, say `USER1` is the currently connected user, and `USER1` has select access to `USER2's EMP` table. Without a synonym, `USER1` must select from `USER2's EMP` table, as follows:

```
SQL> select * from user2.emp;
```

Assuming it has the `CREATE SYNONYM` system privilege, `USER1` can do the following:

```
SQL> create synonym emp for user2.emp;
```

Now, `USER1` can transparently select from `USER2's EMP` table:

```
SQL> select * from emp;
```

Dynamically Generating Synonyms

You can also define a synonym as public synonyms, which means that any user in the database has access to the synonym. Sometimes, an inexperienced DBA does the following:

```
SQL> grant all on sales to public;  
SQL> create public synonym sales for mv_maint.sales;
```

Any user that can connect to the database can perform any INSERT, UPDATE, DELETE, or SELECT operation on the SALES table that exists in the MV_MAINT schema.

Displaying Synonym Metadata

The DBA/ALL/USER_SYNONYMS views contain information about synonyms in the database:

```
SQL> select synonym_name, table_owner, table_name, db_link  
from user_synonyms  
order by 1;
```

Dropping a Synonym

Use the DROP SYNONYM statement to drop a private synonym:

```
SQL> drop synonym inv;
```

If it is a public synonym, then you need to specify PUBLIC when you drop it:

```
SQL> drop public synonym inv_pub;
```

8. Managing Sequences:

A sequence is a database object that users can access to select unique integers. A sequence is a mechanism for automatically generating integers that follow a pattern.

- A sequence has a name, which is how it is referenced when the next value is requested
- A sequence is not associated with any particular table or column
- The progression can be ascending or descending
- The interval between numbers can be of any size
- A sequence can cycle when a limit is reached

Creating a Sequence

Here is a summary of the sequence creation options:

- **Name:** Name of the sequence, which is how it is referenced
- **Schema:** Owner of the sequence
- **Maximum Value:** Specifies the maximum value that the sequence can generate. This integer value can have 28 or fewer digits. It must be greater than Minimum Value and Initial. Using Unlimited indicates the maximum value of 1027 for an ascending sequence or -1 for a descending sequence. The default is Unlimited.
- **Minimum Value:** Specifies the minimum value of the sequence. This integer value can have 28 or fewer digits. It must be less than or equal to Initial and less than Maximum Value. Using Unlimited indicates the minimum value of 1 for an ascending sequence or -1026 for a descending sequence. The default is Unlimited.
- **Interval:** Specifies the interval between sequence numbers. This integer value can be any positive or negative integer, but it cannot be 0. It can have 28 or fewer digits. The default value is 1.
- **Initial:** Specifies the first sequence number to be generated. Use this clause to start an ascending sequence at a value greater than its minimum or to start a descending sequence at a value less than its maximum.

- **Cycle Values:** After an ascending sequence reaches its maximum value, it generates its minimum value. After a descending sequence reaches its minimum, it generates its maximum value. If you do not choose this option, an error is returned when you attempt to retrieve a value after the sequence has been exhausted.
- **Order Values:** Guarantees that sequence numbers are generated in the order of request. This clause is useful if you are using sequence numbers as time stamps. Guaranteeing order is usually not important for sequences that are used to generate primary keys. This option is necessary only to guarantee ordered generation if you are using the CACHE option with Oracle Database with Real Application Clusters. If you are not caching, the sequence is in order by default.
- **Cache Options:** Specifies how many values of the sequence the Oracle database pre-allocates and keeps in memory for faster access. This integer value can have 28 or fewer digits. The minimum value for this parameter is 2. For sequences that cycle, this value must be less than the number of values in the cycle. You cannot cache more values than would fit in a given cycle of sequence numbers.

The following command creates a sequence with a starting value of 1,000 and a maximum value of 1,000,000:

```
SQL> create sequence inv_seq2 start with 10000 maxvalue 1000000;
```

You can see information about user's sequences via this query:

```
SQL> select sequence_name, min_value, increment_by, cache_size,  
max_value  
from user_sequences;
```

Using Sequence Pseudocolumns

You can use two pseudo columns to access the sequence's value:

- NEXTVAL
- CURRVAL

You can reference these pseudo columns in any `SELECT`, `INSERT`, or `UPDATE` statements. To retrieve a value from the `INV_SEQ` sequence, access the `NEXTVAL` value, as shown:

```
SQL> select inv_seq.nextval from dual;
```

Now that a sequence number has been retrieved for this session, you can use it multiple times by accessing the `CURRVAL` value:

```
SQL> select inv_seq.currval from dual;
```

The sequence can be accessed directly in the `INSERT` statement. The first time you access the sequence, use the `NEXTVAL` pseudo column:

```
SQL> insert into inv(inv_id, inv_desc) values (inv_seq.nextval,  
'Book');
```

If you want to reuse the same sequence value, you can reference it via the `CURRVAL` pseudo column. Next, a record is inserted into a child table that uses the same value for the foreign key column as its parent primary key value:

```
SQL> insert into inv_lines  
(inv_line_id, inv_id, inv_item_desc)  
values  
(1, inv_seq.currval, 'Tomel');
```

Viewing Sequence Metadata

To extract the DDL for `INV_SEQ` use:

```
SQL> select dbms_metadata.get_ddl('SEQUENCE', 'INV_SEQ') from  
dual;
```

If you want to display the DDL for all sequences for the currently connected user, run this SQL:

```
SQL> select dbms_metadata.get_ddl('SEQUENCE', sequence_name)
from user_sequences;
```

Dropping a Sequence

To drop a sequence, use the `DROP SEQUENCE` statement:

```
SQL> drop sequence inv_seq;
```

9. Quiz:

1. Which of the following statements is correct about indexes? (Choose the best answer).
 - a) An index can be based on multiple columns of a table, but the columns must be of the same datatype.
 - b) An index can be based on multiple columns of a table, but the columns must be adjacent and specified in the order that they are defined in the table.
 - c) An index cannot have the same name as a table, unless the index and the table are in separate schemas.
 - d) None of the above statements is correct.
2. Which of these statements about indexes is incorrect?
 - a) Indexes can limit the amount of data that is returned in the results.
 - b) Indexes are mandatory objects for each column in a table.
 - c) Indexes can help keep results in memory and bring back results faster.
 - d) Without indexes, the entire table is scanned (full table scan) with each DML operation on the table.
3. Why do not place indexes on all tables and column combinations? (Choose the best answer).
 - a) Because indexes are free
 - b) Because they have no impact on performance and space.
 - c) Because indexes are mandatory.
 - d) Because indexes have impact on performance and space.

4. Which of the following statements is not a part of the structure of B-tree index?
- a) Root, which contains entries that point to the next level.
 - b) Branch blocks, which in turn point to blocks at the next level.
 - c) A bitmap segment consisting of a string of bits
 - d) The leaf blocks are doubly linked to facilitate the scanning of the index in an ascending as well as descending order of key values.
 - e) The leaf nodes, which contain the index entries that point to rows in the table.
5. An index is an optionally created database object used primarily to increase query performance.
- a) True
 - b) False
6. Bitmap indexes should not be used if:
- a) The cardinality (the number of distinct values) in the column is low
 - b) The cardinality (the number of distinct values) in the column is high
 - c) The number of rows in the table is high
 - d) The column is used in Boolean algebra operations
7. Multicolumn indexes are especially effective when you often use multiple columns in the WHERE clause when accessing a table?
- a) True
 - b) False
8. What is the type of the index that is created by the following command:
- ```
SQL> CREATE INDEX cust_idx1 ON cust(last_name);
```
- a) Bitmap index
  - b) B-tree index
  - c) Partitioned index
  - d) Memory index.

9. For the same combination of columns; There can be only?
- a) Only two visible indexes
  - b) Only one visible index
  - c) Only three visible indexes
  - d) Only four visible indexes
10. What will Oracle do when trying to rebuild the index online?
- a) Oracle attempts to rollback on the table
  - b) Oracle attempts to commit on the table
  - c) Oracle attempts to acquire a lock on the table
  - d) Oracle attempts to unlock any lock on the table
11. Choose the correct statement after executing DROP VIEW statement to drop a view:
- a) The related data will be deleted
  - b) The related base table will be dropped too
  - c) The view will be dropped alone
  - d) The related triggers will be dropped
12. Which of the following types of database objects can't have synonym?
- a) Tables
  - b) views
  - c) Sequences
  - d) indexes
13. It is recommended that you separate tables and indexes into different tablespaces?
- a) True
  - b) False

14. Which statements are true about Cycle Values of a sequence?

- a) After an ascending sequence reaches its maximum value, you have to drop it.
- b) After an ascending sequence reaches its maximum value, it generates its minimum value.
- c) After a descending sequence reaches its minimum, it generates its maximum value.
- d) After a descending sequence reaches its minimum, you have to drop it.

15. Which statement is false about Cache Options of a sequence?

- a) Specifies how many values of the sequence pre-allocates and keeps in memory for faster access.
- b) This integer value can have 28 or fewer digits.
- c) The minimum value for this parameter is 20
- d) The minimum value for this parameter is 2

16. The following SQL statement is used to display the “INV\_SEQ” sequence DDL statement:

```
select dbms_metadata.get_ddl('SEQUENCE','INV_SEQ') from
dual;
```

- a) True
- b) False

17. What are the two pseudo columns to access the sequence's value?

- a) Forwardval
- b) Nextval
- c) Retrieivedval
- d) Currval

18. Views are stored queries as well as other complex expressions and SQL constructs?

- a) True
- b) False

19. Choose the correct statements that describe the common uses of views (choose the two statements):

- a) Adding privileges to the user
- b) Creation a User Account with password
- c) Provide an interface layer between an application and physical tables.
- d) Hide the complexity of an SQL query from an application.

20. Any user in the database has access to the public synonym?

- a) True
- b) False

| Quiz answers – chapter 9 |      |
|--------------------------|------|
| 1                        | d    |
| 2                        | b    |
| 3                        | d    |
| 4                        | c    |
| 5                        | a    |
| 6                        | b    |
| 7                        | a    |
| 8                        | b    |
| 9                        | b    |
| 10                       | c    |
| 11                       | c    |
| 12                       | d    |
| 13                       | a    |
| 14                       | b, c |
| 15                       | c    |
| 16                       | a    |
| 17                       | b, d |
| 18                       | a    |
| 19                       | c, d |
| 20                       | a    |

## References:

### Books

1. Oracle Database Administrator's Guide, 18c (Oracle University), March 2019, Primary Author: Rajesh Bhatiya, Randy Urbano
2. database-new-features-guide (Oracle University), 18c E88909-01 February 2018, Primary Authors: Tanaya Bhattacharjee, Sunil Surabhi, Mark Baue
3. The Cloud DBA-Oracle: Managing Oracle Database in the Cloud – First Edition, Abhinivesh Jain and Niraj Mahajan. Apress – ISBN-13 (pbk): 978-1-4842-2634-6 ISBN-13 (electronic): 978-1-4842-2635-3

### Web Sites

1. <https://docs.oracle.com/database>
2. <https://docs.oracle.com/en/database/oracle/oracle-database/19/whats-new.html>
3. <http://www.oracle.com/technetwork/database/enterprise-edition/downloads/index.html>